

Watch Together

Product Requirements Document (PRD)

Version: 1.0

Status: MVP Development

Product Type: Web Platform

Primary Users: Couples, friends, and small groups who want to watch videos together remotely

Development Approach: AI-assisted / AI-first development using Antigravity

1. Product Overview

Watch Together is a web-based social viewing platform that allows two or more people to enter a private virtual room, watch a video together in synchronization, communicate through real-time text chat, and interact through live video and audio.

The core experience should feel like sitting together in the same room while watching a movie.

The platform should prioritize:

- Extremely simple room creation
- Accurate video synchronization
- Low-latency communication
- A beautiful, modern interface
- Private rooms
- Mobile and desktop usability
- Easy extensibility for future features

The initial product should focus on videos that users are legally allowed to watch or content that can legally be embedded or streamed.

The platform must **not** host or distribute copyrighted movies without appropriate rights.

2. Product Vision

Vision

Create the simplest and most enjoyable way for two people who are physically apart to feel like they are watching something together.

Product Principle

The product should feel like:

"We are sitting together, even though we are miles apart."

The experience should be more intimate and focused than a normal video-conferencing application.

3. Problem Statement

People often watch movies, shows, YouTube videos, and other content together while being physically separated.

Existing solutions often require users to combine multiple applications:

- One app for the movie
- One app for video calling
- One app for chatting
- Manual coordination for play/pause
- Problems keeping playback synchronized

This creates unnecessary friction.

Watch Together combines these experiences into one synchronized room.

4. Target Users

Primary User

A person who wants to watch content remotely with:

- Their partner
- Their girlfriend/boyfriend
- A close friend

Secondary Users

Small groups who want to:

- Watch videos together
- Chat
- Talk through voice/video

- Share reactions

Initial User Focus

The initial design should optimize heavily for **two-person rooms**.

Group watching can be supported architecturally but does not need to be the primary MVP experience.

5. Core User Journey

New User

1. User opens the website.
 2. User sees the landing page.
 3. User clicks **Create Room**.
 4. A private room is created.
 5. User receives a shareable room link.
 6. User sends the link to another person.
 7. Second person joins the room.
 8. Both users see each other.
 9. One user selects or loads a supported video.
 10. Video begins playing.
 11. Playback remains synchronized.
 12. Users can chat.
 13. Users can enable camera/microphone.
 14. Users watch and communicate together.
-

6. MVP Features

6.1 Landing Page

The landing page should communicate the product immediately.

Example messaging:

Watch Together

Watch. Talk. Laugh. Together.

Primary CTA:

Create a Room

Secondary CTA:

Join a Room

The landing page should be visually modern and romantic/friendly without being overly themed toward couples.

7. Room System

7.1 Create Room

A user can create a private room.

The system generates:

- Unique room ID
- Shareable room URL

Example:

`watchtogether.com/room/7F3K9A`

The room should not require an account for the initial MVP unless authentication is required for security or persistence.

7.2 Join Room

A user can join by opening the room URL.

The user enters a display name before joining.

Example:

"What should we call you?"

`Aahil`

Then:

Join Room

7.3 Room Capacity

MVP target:

2 users

The underlying architecture should avoid hard-coding the system so tightly that supporting more users later becomes impossible.

Future target:

4–8 users

7.4 Room Privacy

Rooms should be private by default.

Anyone possessing the room link can potentially join unless an additional access mechanism is implemented.

Future security features may include:

- Room passwords
 - Invite approval
 - Host controls
 - Room expiration
 - Kick/ban controls
-

8. Video Player

The video player is the central feature of the application.

Required Controls

- Play
- Pause
- Seek
- Timeline
- Volume
- Mute
- Fullscreen
- Playback speed where technically supported

Synchronization

The following actions must synchronize between participants:

- Play
- Pause
- Seek
- Current playback position
- Video loading state where applicable

Example:

User A presses pause at:

00:52:31

The server broadcasts the event.

User B's player should pause at approximately the same timestamp.

9. Video Synchronization Architecture

The synchronization system should use real-time communication.

Recommended technology:

WebSockets / Socket.IO

Example event:

```
{
  "type": "PLAY",
  "roomId": "7F3K9A",
  "timestamp": 382.42,
  "userId": "user123"
}
```

Example pause event:

```
{
  "type": "PAUSE",
  "roomId": "7F3K9A",
```

```
"timestamp": 382.42,  
"userId": "user123"  
}
```

Example seek event:

```
{  
  "type": "SEEK",  
  "roomId": "7F3K9A",  
  "timestamp": 531.12,  
  "userId": "user123"  
}
```

10. Synchronization Requirements

The synchronization system must:

- Prevent infinite event loops
- Handle users joining an active room
- Correct newly joined users to the current timestamp
- Handle network delay
- Handle temporary disconnections
- Prevent clients from continuously fighting over playback state

Host Model

For MVP, implement a simple **host-controlled synchronization model**.

The room creator becomes the host.

Host actions determine the canonical playback state.

Future versions can support:

- Shared control
 - Automatic host transfer
 - Permission-based controls
-

11. Joining an Active Room

When a user joins an existing room, they should receive the current room state.

Example:

```
{  
  "video": "example-video",  
  "playing": true,  
  "timestamp": 1832.4  
}
```

The new client should initialize its player accordingly.

12. Real-Time Chat

Each room should contain a real-time chat panel.

Users can send messages without refreshing the page.

Example:

```
Aahil:  
this scene 😭  
  
GF:  
HAHAHAHA 😂
```

Chat Requirements

Support:

- Text messages
- Username
- Timestamp
- Message ordering
- Automatic scrolling to latest messages

Future features:

- Emoji

- GIFs
 - Reactions
 - Replying
 - Message deletion
-

13. Video Calling

Users should be able to see and hear each other while watching.

Technology

Use:

WebRTC

The actual media connection should preferably be peer-to-peer for the two-person MVP.

The application server should handle signaling.

14. Video Call Requirements

Each participant should have:

- Camera toggle
- Microphone toggle
- Camera preview
- Mute status indicator
- Connection status

UI should support:

Compact Mode

Small floating video bubbles over the movie.

Expanded Mode

A larger video-call panel beside the movie.

The movie must remain the primary visual focus.

15. Audio Requirements

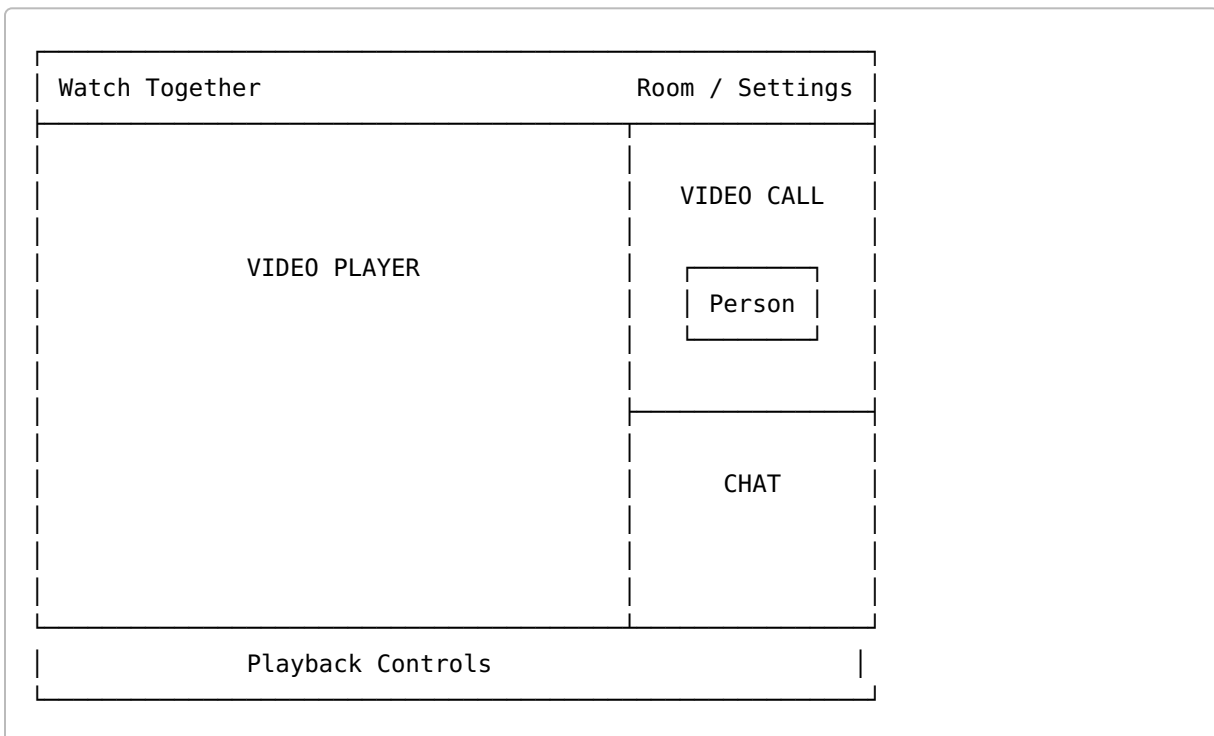
Audio should support:

- Microphone on/off
- Remote participant audio
- Camera on/off without disabling microphone
- Browser permission handling

The application must clearly explain when camera/microphone permissions are required.

16. Main Room Layout

Desktop layout:



On mobile, panels should stack intelligently.

17. UI / UX Requirements

The interface should feel:

- Modern
- Minimal
- Smooth
- Warm
- Premium
- Personal
- Fast

Avoid:

- Overly complicated dashboards
- Excessive buttons
- Corporate design
- Clutter
- Huge amounts of text

Use subtle animations.

Buttons should provide visual feedback.

Loading states should always be visible when an operation takes time.

18. Responsive Design

The application must support:

- Desktop
- Laptop
- Tablet
- Mobile

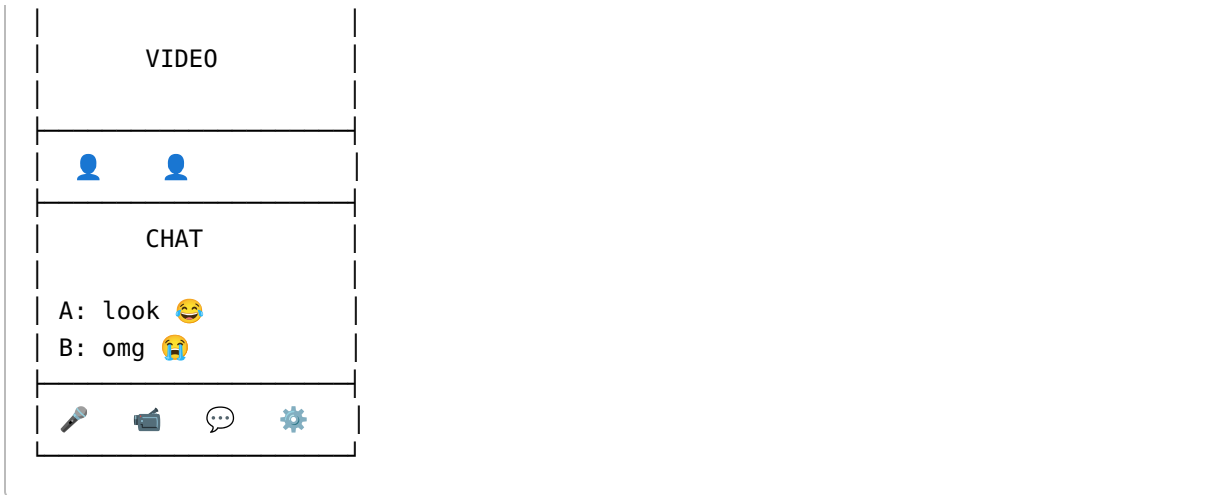
Desktop should prioritize the movie and chat/video layout.

Mobile should use a vertically stacked experience.

Example mobile layout:



Watch Together



19. User Presence

The room should display whether participants are:

- Online
- Connecting
- Disconnected
- Reconnecting

Example:

● Sarah — Online

If a participant disconnects:

○ Sarah — Reconnecting...

20. Connection Handling

The application must gracefully handle:

- Internet interruption
- WebSocket disconnection
- WebRTC failure
- Browser refresh
- Temporary reconnection

The user should see:

Reconnecting...

rather than a broken interface.

21. Database

Recommended database:

PostgreSQL through Supabase

Potential tables:

users

```
id
display_name
avatar_url
created_at
```

rooms

```
id
room_code
host_user_id
created_at
expires_at
```

room_members

```
id
room_id
user_id
joined_at
```

messages

```
id
room_id
user_id
```

```
message
created_at
```

The MVP may use anonymous/temporary user identities if persistent accounts are not required.

22. Authentication

Authentication is not mandatory for the first MVP.

Preferred initial approach:

Guest identity

User enters a temporary display name.

Later versions can introduce:

- Google login
- Email login
- User profiles
- Saved rooms
- Watch history

23. Recommended Technology Stack

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS
- shadcn/ui

Backend

- Node.js
- Next.js server functionality where appropriate
- WebSockets / Socket.IO

Database

- Supabase

- PostgreSQL

Video Calling

- WebRTC

Hosting

- Vercel
 - Supabase
 - WebSocket-capable backend where required
-

24. Suggested Project Architecture

```
watch-together/  
|  
├─ app/  
|   ├─ page.tsx  
|   ├─ create/  
|   ├─ join/  
|   └─ room/  
|  
├─ components/  
|   ├─ VideoPlayer/  
|   ├─ Chat/  
|   ├─ VideoCall/  
|   ├─ RoomHeader/  
|   ├─ Participants/  
|   └─ Controls/  
|  
├─ hooks/  
|   ├─ useRoom.ts  
|   ├─ useChat.ts  
|   ├─ useVideoSync.ts  
|   └─ useWebRTC.ts  
|  
├─ lib/  
|   ├─ socket.ts  
|   ├─ webrtc.ts  
|   ├─ supabase.ts  
|   └─ room.ts  
|  
└─ types/  
    └─ room.ts
```

```
|   |   | chat.ts  
|   |   | video.ts  
|   |   |  
|   |   | server/  
|   |   | | websocket/  
|   |   | | signaling/
```

The exact architecture may be adjusted by the AI agent when implementation constraints require it.

25. API / Realtime Events

The realtime layer should define explicit event types.

Example:

```
ROOM_JOIN  
ROOM_LEAVE  
ROOM_STATE  
USER_JOINED  
USER_LEFT  
  
VIDEO_PLAY  
VIDEO_PAUSE  
VIDEO_SEEK  
VIDEO_STATE_REQUEST  
VIDEO_STATE_RESPONSE  
  
CHAT_MESSAGE  
  
WEBRTC_OFFER  
WEBRTC_ANSWER  
WEBRTC_ICE_CANDIDATE
```

Events should be strongly typed using TypeScript.

26. Security Requirements

The application must:

- Validate room IDs
- Validate incoming messages

- Sanitize chat messages
- Never trust client-provided permissions
- Rate-limit chat/realtime events where practical
- Avoid exposing private server credentials
- Never expose Supabase service-role keys to clients
- Secure WebSocket connections in production
- Use HTTPS
- Handle invalid rooms safely

The system should prevent a malicious client from easily impersonating another user.

27. Privacy

The product should minimize collected data.

For MVP:

- Display name
- Room information
- Chat messages while retained
- Technical connection information required for operation

Do not collect unnecessary personal information.

Camera and microphone access must only occur after appropriate user interaction/browser permission.

28. Copyright / Content Policy

The application must not provide unauthorized copies of copyrighted movies.

The system should support:

- User-owned content
- Public-domain content
- Properly licensed content
- Legally embeddable third-party content

The product should not scrape or bypass DRM from streaming services.

29. Performance Requirements

Target:

- Fast initial page load
- Realtime chat latency: preferably under 500 ms
- Playback synchronization should feel effectively simultaneous
- Video call should have acceptable latency for normal internet connections
- Reconnection should happen automatically when possible

The room should remain usable on average consumer internet connections.

30. Error States

The system must handle errors gracefully.

Examples:

Invalid Room

This room doesn't exist or has expired.

Video Loading Error

We couldn't load this video.

Camera Permission Denied

Camera access is blocked. You can continue without video.

Microphone Permission Denied

Microphone access is blocked.

Connection Error

Connection lost. Reconnecting...

Full Room

This room is currently full.

31. MVP Definition of Done

The MVP is considered complete when:

1. A user can create a room.
 2. Another user can join through a link.
 3. Both users can see each other's presence.
 4. A supported video can be loaded.
 5. Play/pause is synchronized.
 6. Seeking is synchronized.
 7. New users receive the current playback state.
 8. Users can send real-time chat messages.
 9. Users can enable/disable cameras.
 10. Users can enable/disable microphones.
 11. Two users can communicate through WebRTC.
 12. The application reconnects after temporary network issues.
 13. The interface works on desktop and mobile.
 14. No major console errors remain.
 15. Production deployment works securely.
-

32. Future Features

These should NOT block MVP development.

Social

- Reactions
- Emoji bursts
- GIFs
- Shared stickers
- Typing indicators

Couples Features

- Shared watchlist
- Date-night mode
- Shared playlists
- "Pick our next movie"
- Couple profiles
- Shared memories

Groups

- 4–8 participants

- Host permissions
- Participant moderation
- Group reactions

Content

- YouTube integration
- Public-domain movie library
- User uploads
- Multiple video sources

Accounts

- Profiles
- Watch history
- Saved rooms
- Friends
- Favorites

Premium Features

Potential future options:

- Higher participant limits
- Better video quality
- Custom rooms
- Custom themes
- Cloud storage

33. Analytics

Only basic anonymous analytics should initially be collected.

Potential metrics:

- Rooms created
- Rooms joined
- Average session duration
- Video playback failures
- WebRTC connection failures
- Chat usage
- Returning users

Do not prioritize analytics over the core user experience.

34. Design Direction

The visual identity should feel like a combination of:

modern streaming service + private hangout + video call

Suggested visual characteristics:

- Dark interface
- Large video area
- Soft rounded components
- Subtle gradients
- Smooth transitions
- Minimal borders
- High-quality typography
- Floating controls

The interface should look polished enough to resemble a real consumer product rather than a developer demo.

35. AI Development Requirements

Antigravity should follow these rules during development:

Rule 1 — Build incrementally

Do not attempt to create the entire product in a single implementation step.

Development should proceed feature by feature.

Rule 2 — Keep functionality working

After implementing each major feature:

- Run the application
- Test the feature
- Fix errors
- Check browser console
- Check server logs

Rule 3 — Strong typing

Use TypeScript throughout the application.

Avoid unnecessary `any`.

Rule 4 — Reusable components

Avoid putting the entire application into one giant component.

Rule 5 — Environment variables

Secrets must use environment variables.

Never hard-code:

- API keys
- Database credentials
- Private tokens
- Service-role keys

Rule 6 — Error handling

Every network operation should have appropriate error handling.

Rule 7 — Mobile support

Mobile layout must be considered from the beginning instead of being added at the end.

36. Recommended AI Implementation Sequence

Antigravity should implement the product in this order:

Step 1

Initialize the project and install the required dependencies.

Step 2

Build the landing page.

Step 3

Build room creation and joining.

Step 4

Build the complete room UI using mocked data.

Step 5

Implement realtime room presence.

Step 6

Implement realtime chat.

Step 7

Implement video playback.

Step 8

Implement synchronized playback.

Step 9

Implement WebRTC signaling.

Step 10

Implement video/audio calls.

Step 11

Implement reconnection and failure handling.

Step 12

Perform full UI polish and responsiveness work.

Step 13

Perform security review.

Step 14

Perform production deployment.

Step 15

Perform end-to-end testing with two browsers/devices.

37. Testing Requirements

The AI agent must test at minimum:

Room

- Create room
- Join room
- Invalid room
- Refresh page
- Leave room

Chat

- Send message
- Receive message
- Multiple messages
- Empty message handling
- Long message handling

Video

- Play
- Pause
- Seek
- Join while video is playing
- Network interruption
- Different browser timing

Video Call

- Camera enable
- Camera disable
- Microphone enable
- Microphone disable
- Connection failure
- Reconnection

Responsive

Test:

- Desktop Chrome
- Mobile Chrome

- Mobile Safari where possible
-

38. Success Criteria

The MVP succeeds when two people can:

Open the website → create a private room → share the link → join together → load a supported video → watch it in sync → chat → turn on their cameras and microphones → stay connected for an entire viewing session.

The experience should require little or no technical knowledge from the users.

39. Final Product Goal

The final experience should not feel like several unrelated tools stitched together.

It should feel like **one cohesive product**.

A user should be able to enter a room and immediately understand:

"This is where we watch together."

The movie should remain the main focus.

Chat and video should enhance the experience rather than overwhelm it.

The first release should prioritize **reliability, synchronization, simplicity, and visual quality** over having a huge number of features.

40. One-Sentence Product Definition

Watch Together is a private social viewing platform that lets people watch videos in sync while chatting and video calling inside the same virtual room.